

Perception Server Explained

Simple explanation of the backend server, Docker, and Google Cloud Run setup.

Goal: the Flutter app will send an OFP PDF to a server. The server will receive the file, check it, and later parse it to produce the fuel analysis.

1. Why We Need A Server

Your Flutter app runs on the user's device. It is good for the interface: login screen, upload screen, analysis screen, buttons, and charts.

The server runs separately on the internet. It is good for heavier or protected work: receiving PDFs, parsing OFP text, applying business rules, saving history, and checking authorization.

2. Current Project Structure

The app and server live in the same repository:

perceptionv1/	
lib/	Flutter app code
android/	Android project
ios/	iOS project
pubspec.yaml	Flutter dependencies
server/	Backend API server
main.py	FastAPI server code
requirements.txt	Python dependencies

Dockerfile

Container instructions

README.md

Local run and deploy notes

Keeping both parts together is normal at this stage. It makes development easier because app and API changes are in one place.

3. How The Server Works

The server is built with **FastAPI**, a Python framework for creating APIs.

Right now it exposes two endpoints:

- GET /health: checks if the server is alive.
- POST /analyze: receives one PDF file.

Health Check

```
GET /health
```

Response:

```
{ "status": "ok" }
```

PDF Upload

```
POST /analyze
```

The Flutter app will send a multipart form request. The PDF field name is `file`.

The server currently checks:

- the uploaded file name ends with `.pdf`
- the file is not empty
- the file is not larger than 25 MB

For now the response only confirms that the PDF arrived:

```
{

  "status": "received",

  "filename": "example.pdf",

  "content_type": "application/pdf",

  "size_bytes": 123456,

  "message": "PDF received. Parsing logic will be added next."

}
```

4. What Docker Is

Docker packages the server and everything it needs into a container. A container is like a small isolated environment that can run the same way on your computer and on Google Cloud Run.

Without Docker, we must manually prepare Python, install packages, and start the server on every machine. With Docker, the instructions are written once in the `Dockerfile`.

What The Dockerfile Does

```
FROM python:3.12-slim
```

Start from a small Python image.

```
WORKDIR /app
```

Use `/app` as the working folder inside the container.

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

Copy the dependency list and install FastAPI, Uvicorn, and file upload support.

```
COPY main.py .
```

Copy the server code into the container.

```
CMD exec uvicorn main:app --host 0.0.0.0 --port ${PORT:-8080}
```

Start the API server. Cloud Run provides the `PORT` variable automatically.

5. What Google Cloud Run Is

Cloud Run is a Google service that runs containers on the internet. You give it the server container, and Google handles the machine, HTTPS URL, scaling, and restarts.

If nobody is using the API, Cloud Run can scale down. If many users send PDFs, Cloud Run can start more instances.

6. Local Development Flow

When developing locally, you can run the server on your machine:

```
cd server
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate

pip install -r requirements.txt

uvicorn main:app --reload --port 8080
```

Then test it:

```
curl http://127.0.0.1:8080/health
```

Upload a PDF:

```
curl -F "file=@/path/to/ofp.pdf" http://127.0.0.1:8080/analyze
```

7. Deployment Flow

Later, from the project root, the server can be deployed to Cloud Run:

```
gcloud run deploy perception-ofp-api \

--source server \

--region europe-west1 \

--allow-unauthenticated
```

The `--source server` part tells Google to deploy only the backend folder, not the whole Flutter app.

8. Authorization In Simple Terms

Authorization means deciding who can use the app and who can call the server.

A good approach for this app is Firebase Authentication or Google Cloud Identity Platform:

1. User logs in inside the Flutter app.
2. Flutter receives an ID token after login.
3. Flutter sends the token to the server in every protected request.
4. The server checks the token before accepting the PDF.

The request would look like this:

```
Authorization: Bearer <id-token>
```

At the beginning we can keep the server open for testing. Later we add token verification before processing `/analyze`.

9. Next Steps

1. Connect Flutter upload to `POST /analyze`.
2. Return mock analysis JSON from the server.
3. Replace mock data with real PDF text extraction.
4. Add OFP parsing logic.
5. Add authorization token checks.
6. Deploy to Cloud Run.

Simple mental model: Flutter is the cockpit interface. The server is the engine room. Docker is the box that packages the engine room. Cloud Run is where Google runs that box online.